

CH11 容器化与云原生基础

考试复习笔记

来源：开源操作系统课件（53 页） 日期：2026 年 6 月 23 日

1 为什么需要容器化 [p.8–10]

1.1 多实例部署的挑战 [p.8–9]

传统方式每台服务器需手工：安装系统依赖 → 创建运行用户 → 安装 JDK/Nginx/MySQL/Redis → 上传 JAR → 配置环境变量 → 创建 systemd 服务 → 启动验证。

三大痛点：

1. 环境不一致：同一 JAR 包在不同服务器表现不同（JDK 版本、依赖差异）
2. 依赖难复制：每台服务器精确安装所有依赖 = 灾难
3. 扩缩容困难：扩容一台 30min 2h，缩容无法快速释放
4. 版本回滚复杂：回滚耗时与升级相当

2 什么是容器 [p.11–14]

2.1 容器定义 [p.11]

容器是在 Linux 内核之上，利用 **namespace**（隔离视图）、**cgroups**（资源限制）、联合文件系统等机制运行的一组受隔离、受限制的进程。

2.2 容器核心理念 [p.12]

- 容器中运行的是**进程**（如 `java -jar app.jar`），不是完整机器
- 容器依赖**镜像**启动（镜像 = 应用交付制品）
- 容器化 = 以镜像和容器为核心的构建、交付、部署方式

2.3 容器 vs 虚拟机 [p.14]

维度	虚拟机	容器
隔离级别	硬件级 (Hypervisor)	进程级 (Linux 内核)
操作系统	每个 VM 独立 Guest OS	所有容器共享 Host 内核
启动速度	分钟级	秒级
资源开销	GB 级	MB 级
镜像大小	GB 级	MB~ 百 MB
密度	几 ~ 几十个/台	几百 ~ 几千个/台

2.4 容器技术发展 [p.15]

chroot → Linux namespace/cgroups → LXC → **Docker** (标准化镜像构建 + 仓库 + 开发者体验)
→ OCI/containerd/Kubernetes 云原生生态

3 Docker [p.19–35]

3.1 Docker 架构组件 [p.19]

组件	功能
Docker Client	用户 CLI
Docker Daemon	后台服务, 管理镜像和容器
containerd	容器运行时, 管理容器生命周期
runc	底层运行时, 调用 Linux 内核
Registry	镜像仓库 (Docker Hub / 私有仓库)

3.2 镜像 vs 容器 [p.20]

镜像 (Image) = 类 (Class): 只读模板, 定义运行所需的一切。

容器 (Container) = 对象 (Object): 镜像的运行实例, 有独立状态。

3.3 Dockerfile 指令 [p.23] (重要考点)

指令	作用	示例
FROM	指定基础镜像	FROM openjdk:17-jdk-slim
WORKDIR	设置工作目录	WORKDIR /app
COPY	复制文件到镜像	COPY target/app.jar app.jar
RUN	构建时执行命令	RUN apt-get update && apt-get install -y curl
ENV	设置环境变量	ENV JAVA_OPTS="-Xmx1g"
EXPOSE	声明暴露端口	EXPOSE 8080
CMD	容器启动命令 (可覆盖)	CMD ["java", "-jar", "app.jar"]
ENTRYPOINT	容器入口 (不可覆盖)	ENTRYPOINT ["java", "-jar", "app.jar"]

CMD vs ENTRYPOINT:

- CMD: 可被 docker run 时覆盖
- ENTRYPOINT: 固定入口, 不会被覆盖
- 组合使用: ENTRYPOINT 固定命令, CMD 提供默认参数

3.4 Docker 常用命令 [p.22,35]

```

docker pull nginx:latest          # 拉取镜像
docker images                     # 查看本地镜像
docker run -d -p 80:80 nginx      # 后台运行容器
docker ps / ps -a                 # 查看运行中/所有容器
docker stop/start/rm <ID>        # 停止/启动/删除容器
docker logs <ID>                  # 查看日志
docker exec -it <ID> bash         # 进入容器
docker volume create myvol        # 创建数据卷
docker run -v myvol:/data image   # 挂载数据卷

```

3.5 make 与 makefile [p.29-33]

makefile 语法: 由依赖关系和规则组成。

```

myapp: main.o 2.o 3.o             # 依赖关系
    gcc -o myapp main.o 2.o 3.o # 规则
main.o: main.c a.h
    gcc -c main.c

```

4 容器编排 [p.42-52]

4.1 为什么需要编排 [p.42-43]

手动管理多容器的问题: 启动顺序、网络配置、环境变量、数据持久化、日志管理、扩容缩容。

4.2 Docker Compose [p.44–47]

单机多容器编排，通过 YAML 文件定义服务。

```
version: "3.8"
services:
  petstore:
    image: petstore:v2.0
    ports: ["8080:8080"]
    depends_on: [mysql, redis]
  mysql:
    image: mysql:8.0
    volumes: ["mysql-data:/var/lib/mysql"]
volumes:
  mysql-data:
```

常用命令: `docker compose up -d / ps / logs -f / down / up -d --scale petstore=5`

4.3 Kubernetes [p.48–52]

当容器数量和服务增多，Compose 不够用（只能管理单机）。Kubernetes 提供：

- 自动调度：决定容器运行在哪台机器
- 自愈能力：容器崩溃自动重启，机器宕机自动迁移
- 服务发现：内置 DNS
- 负载均衡：自动分发请求
- 滚动更新：逐批替换，出问题自动回滚
- 自动扩缩容：根据 CPU/内存/QPS 自动调整

核心思想——声明式期望状态 [p.49–50]：

用户声明期望状态（replicas: 3）→ Kubernetes 持续观察真实状态 → 发现不一致 → 自动调整（创建/删除/重启）→ 真实状态接近期望状态。